

# 實習生月報

## PDO 資料路徑重建

CoE / SDO 交易重建

PDO Mapping / Assignment 關聯整理

SyncManager / FMMU 資料路徑分析

報告期間

2026.08

MONTHLY REPORT

# 本月完成摘要

01

SDO TRANSACTION



CoE / SDO transaction reconstruction

02

PDO MAPPING



Mapping / Assignment relationship reconstruction

03

SM / FMMU



Process data path analysis

04

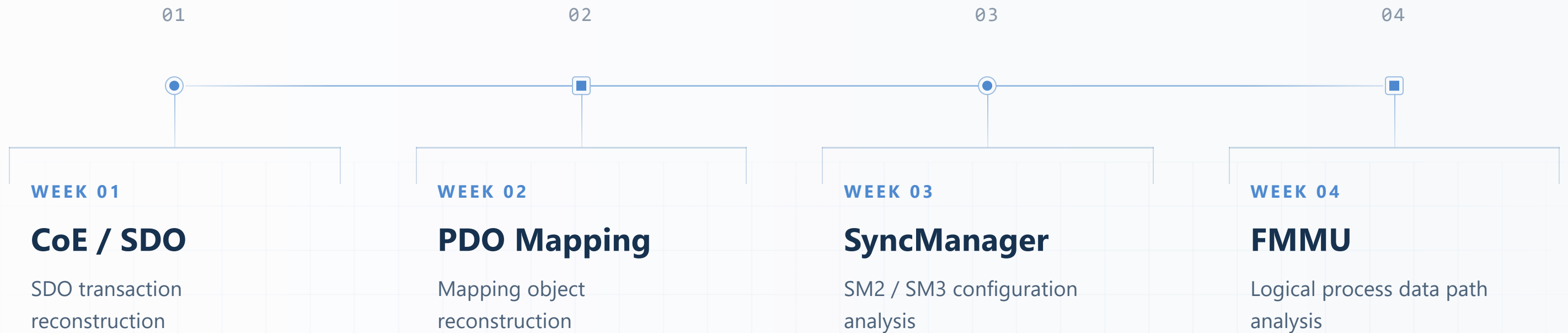
ADDRESS BINDING



Logical region to configured slave binding



# 本月工作大綱



01

# CoE / SDO

## 交易重建

從 EtherCAT Mailbox / CoE 封包還原 SDO object access

MAILBOX — CoE — SDO

# EtherCAT Command / Addressing

## 01 DEVICE / POSITION

### Auto Increment

利用 position 遞增語意逐站命中 slave

## 02 DEVICE / NODE

### Configured Address

以 configured station address 定向存取單一 slave

## 03 DEVICE / ALL

### Broadcast

對所有 slave 的相同 local offset 共同操作

## 04 LOGICAL / FMMU

### Logical Address

以 logical address 經 FMMU 映射至 local memory

## 04 / LOGICAL / FMMU

### Logical Address

指令 LRD / LWR / LRW

地址欄位 32-bit Logical Address

#### 核心機制

Master 存取的是 logical address，而不是直接指定某個 slave 的 ADP / ADO。邏輯位址範圍會經由 FMMU 映射到對應 slave 的 local memory，進而形成 process data 的資料路徑。

#### 補充說明

這個 family 的重點已從「命中哪個 slave」，轉成「logical region 如何被映射到實際資料區塊」。

#### 封包觀察

分析時要關注：

- logical address 範圍
- 對應 FMMU 映射結果
- 最終落到哪些 slave / local memory 區段
- 是否為 PDO / cyclic process data path

#### 典型用途

- PDO / process data 週期性讀寫
- 邏輯位址空間的 cyclic access
- 與 FMMU / SyncManager 分析串接

#### 結論

Logical Address 是從「資料路徑與映射結果」出發的存取方式。

# Startup Register Access

ANALYZER USED REGISTERS

01 STATION ADDRESS

0x0010 Configured Station Address

透過 APWR 寫入 Configured Station Address，並將該值綁定到由 Auto Increment Address 推導出的 topology position。

APWR ADO = 0x0010 ↓ Configured Station Address

02 AL STATE

0x0120 AL Control

擷取 Master 寫入的 AL Control 值，後續整理為 Slave 的 Control waveform。

0x0120 AL Control Master requested state

0x0130 AL Status

擷取 Slave 回報的 AL Status 值，後續整理為 Slave 的 Status waveform。

0x0130 AL Status Slave current state

03 SII EEPROM IDENTITY

0x0502 EEPROM Control / Status

透過 APWR 發出 EEPROM Read 操作，並取得本次欲讀取的 EEPROM Word Address。

APWR ADO = 0x0502 ↓ EEPROM Read Command + EEPROM Word Address

0x0508 EEPROM Data

接續前一筆 EEPROM Read，透過 APRD 取得 EEPROM Data，再依先前記錄的 EEPROM Word Address 判斷資料用途。

APRD ADO = 0x0508 ↓ EEPROM Data

EEPROM Word 0x0008  
Vendor ID

EEPROM Word 0x000A  
Product Code

COMMON OPERATIONS

01 配置 Slave Station Address

APWR  
ADO 0x0010  
→ Configured Station Address

02 記錄 AL Control

0x0120  
→ Control waveform

03 記錄 AL Status

0x0130  
→ Status waveform

04 讀取 Slave Identity

APWR 0x0502  
→ EEPROM Read  
APRD 0x0508  
→ EEPROM Data  
0x0008 → Vendor ID  
0x000A → Product Code

ADDRESS SPACE

ESC Register Address 與 SII EEPROM Word Address  
屬於不同 address space。

0x0502 / 0x0508 → ESC Register

0x0008 / 0x000A → SII EEPROM Word

# Auto Increment → Slave Discovery

[STEP 01]

[STEP TITLE]

[STEP BODY]

[STEP 02]

[STEP TITLE]

[STEP BODY]

[STEP 03]

[STEP TITLE]

[STEP BODY]

[STEP 04]

[STEP TITLE]

[STEP BODY]

[STEP 05]

[STEP TITLE]

[STEP BODY]

[RESULT TITLE]

[RESULT ITEM]

[RESULT ITEM]

[RESULT ITEM]

[RESULT ITEM]

[RESULT ITEM]





# 分析問題與目標

## 01 PROBLEM

### 封包解析提供的是獨立的 **CoE / SDO Record**

單一 Datagram 無法直接表示一次完整的 SDO object access，  
Request 與 Response 必須跨封包建立關聯。

- Request / Response 分散
- Index / SubIndex 資訊分散
- Result / Abort 狀態需要整合

## 02 GOAL

### 重建完整的 **SDO Transaction**

將 Mailbox / CoE / SDO 欄位重新組合，  
建立可供後續 PDO Mapping 分析使用的 transaction-level model。

- Request ↔ Response
- Index / SubIndex
- Access Direction
- Result / Abort

## INPUT

Mailbox / CoE Datagram

## PROCESS

Transaction Reconstruction

## OUTPUT

SDO Transaction Model





# SDO Transaction Reconstruction Flow



# EtherCAT Spec 重點整理

01

KEY CONCEPT

## Core concept or mechanism

Summarize the core mechanism, state transition, or protocol rule that defines the behavior described in the specification. Note what the device or analyzer should observe when this rule is applied.

02

IMPORTANT FIELDS

## Registers / objects / parameters

Record register addresses, object indices, data types, access rules, and parameter constraints. Highlight which fields are read-only, writable, or dependent on operating state.

03

DATA FLOW

## How data moves through the system

Trace how the request, datagram, mapped memory, and application data move through the system. Mark the boundaries where timing, ownership, or synchronization affects the result.

04

ENGINEERING NOTE

## Implementation or analyzer relevance

Connect the specification detail to source code, packet analysis, validation steps, or observable system behavior. Record the implementation risk or evidence that should be checked.



# EtherCAT 圖解分析

FIGURE / 04



## [ Diagram Canvas ]

Class Diagram / Architecture Diagram / Data Path Figure

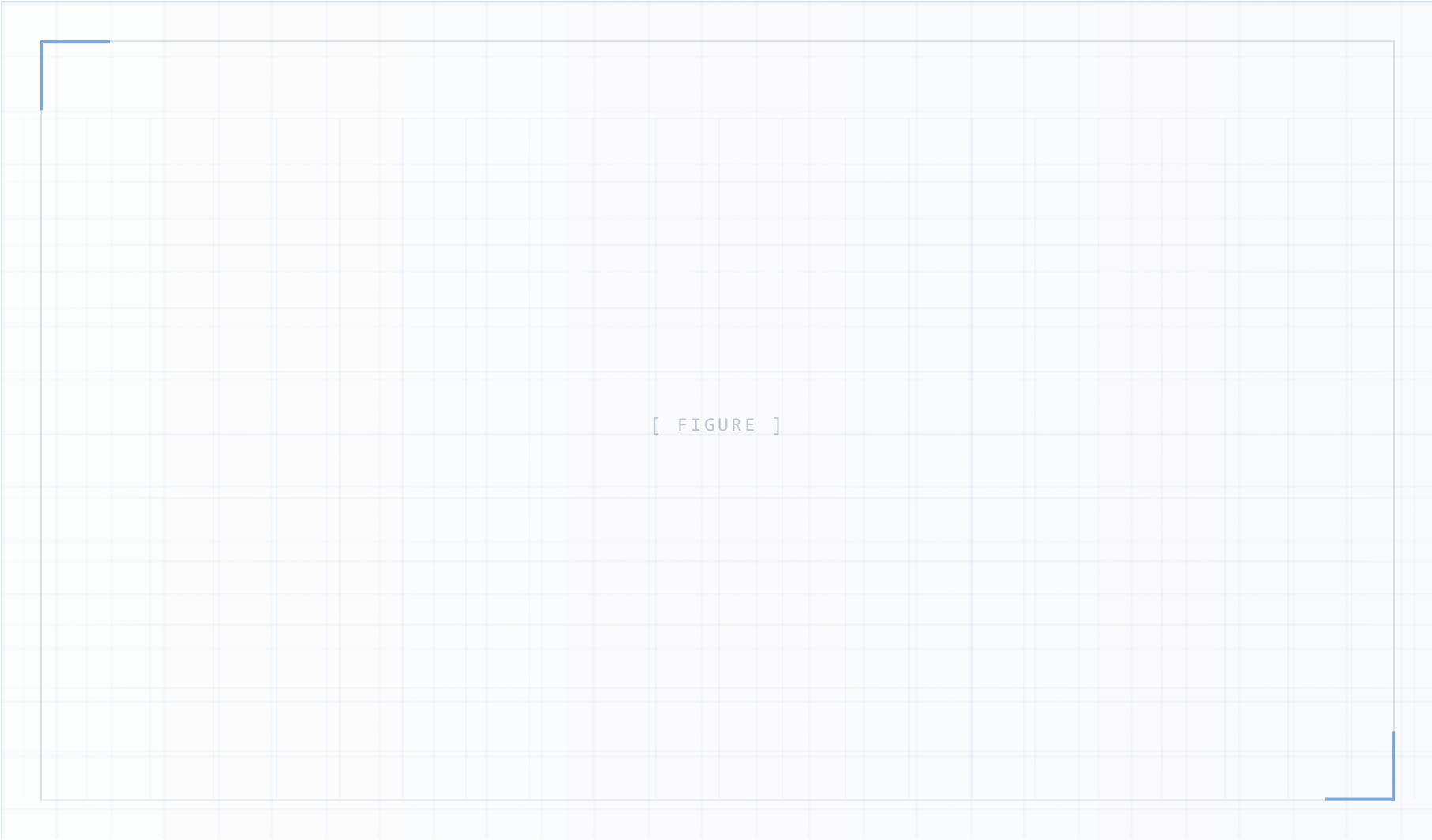
REPLACE WITH IMG / SVG / FIGURE / HTML DIAGRAM

### FOCUS POINTS

- 01 Diagram Purpose**  
Explain the main relationship between modules.
- 02 Key Components**  
Highlight the major classes or architecture blocks.
- 03 Reading Order**  
Describe how to read the diagram efficiently.
- 04 Implementation Relevance**  
Connect the diagram to the current development work.



# EtherCAT 資料路徑概覽



## KNOWLEDGE POINTS

01

### FMMU

Logical Address  
→ ESC local

02

### SyncManager

Consistent data  
exchange control

03

### Process RAM

Stores mapped  
process data

04

### DATA PATH

Logical Address → FMMU →  
SM → RAM → Application